SQL Script Zen City (Q1 2022)

This script provides a comprehensive, end-to-end technical foundation for the Q2 Optimization Project, covering data validation, customer profile, efficiency, and predictive diagnostics.

- INDEX

I. DATA PREPARATION & CLEANING pg 3

- TABLE RENTALS:

1. CHECKING POTENTIAL PRIMARY KEY.
2. CHECKING MISSING VALUES.
3. ENSURE THAT trip_id HAS NO NULL VALUES.
4. VERIFY IF THERE ARE NULL VALUES IN start_station_id.
5. CHECKING NULL VALUES IN end_station_id.
6. CHECKING IF THE STATION WITHOUT ID: Springfest 2022 ITS PRESENT AT TABLE station_info.
7. Due to a single null value in the end_station_id column, we decided to analyze it independently of end_station_name to ensure result accuracy.
8. IDENTIFYING ROW COUNT DISCREPANCIES BETWEEN end_station_id AND end_station_name.
9. CHECKING Rainey STATION IN station_info TABLE.
10. IDENTIFY STATIONS IN THE RENTALS TABLE THAT ARE NOT RECORDED IN THE STATION_INFO TABLE.

- TABLE STATION_INFO

1. CHECKING POTENTIAL PRIMARY KEY.
2. CHECKING MISSING VALUES.
3. IDENTIFY STATIONS IN THE RENTALS TABLE THAT ARE NOT RECORDED IN THE STATION_INFO TABLE.
4. ANALYSING STATIONS status.
5. STATUS OF THE STATIONS FROM RENTALS TABLE.
6. ANALYSING property_type.
7. CHECKING RELATIONS BETWEEN name AND alternate_name.
8. TOTAL STATIONS PER council_district.
9. ANALYSING power_type.
10. ANALYSING number_of_docks.

II. DEMAND PATTERNS AND CUSTOMER PROFILING pg 11

1. START STATIONS IN rentals TABLE (start_station_id).
2. TOP START STATIONS BY USAGE PERCENTAGE.
3. TOP END STATIONS BY USAGE PERCENTAGE.
4. Ensure that start_stations are also recorded as an end station.
5. ANALYSING subscriber_type.
6. TYPES AND SUM OF SUBSCRIBERS.
7. Bike_type ANALYSIS BY USAGE PERCENTAGE.
8. BIKE ID ANALYSIS: COUNT OF UNIQUE BIKES AND TOTAL TRIPS.

9. ANALYSIS OF LOW-USAGE BIKES (1-2 Trips).
10. CALCULATING THE AVERAGE BIKE USAGE.
11. EXPLORATION OF TRIPS DURATION (duration_minutes).
12. ANALYSING MOST POPULAR start_stations AND THEIR CHARACTERISTICS.
13. ANALYSING MOST POPULAR end_stations AND THEIR CHARACTERISTICS.
14. POPULAR ROUTES.
15. STATION LOCATION FROM MOST POPULAR ROUTES.
16. Query for BigQuery Geo Viz mapping of the Top 5 Popular Routes and location from all the stations.

III. DEMAND AND TEMPORAL ANALYSIS pg 24

1. TOTAL RENTALS PER MONTH.
2. TOTAL RENTALS BY WEEK/WEEKEND PER MONTH.
3. TOTAL RENTALS BY DAY OF THE WEEK + WEEKDAY TYPE.
4. TOTAL OF RENTALS: WEEKDAY TYPE PER MONTH.
5. TOTAL RENTALS PER HOUR (PEAK HOURS).
6. TOTAL RENTALS WEEK/WEEKEND x HOUR.
7. TOTAL RENTALS x POPULAR STATIONS x RUSH/PEAK HOURS (10hr - 19hr).
8. start_stations: AVERAGE WEEK/WEEKEND DAY PER DOCK.
9. end_stations: AVERAGE WEEK/WEEKEND DAY PER DOCK.
10. TRIPS PER DOCK start_station.
11. TRIPS PER DOCK end_station.
12. BIKE ROTATION ANALYSIS PER HOUR IN POPULAR ROUTES.
13. Calculates the time-based station load, measuring average trip departures per dock, with imputation for missing capacity data.
14. Query for the average hourly load during weekday peak hours, including the imputation (normalization) of missing dock capacity for stations lacking station information.

IV. PREDICTION MODEL (APRIL 1ST, STATION 2498) pg 36

1. Data Collection and Preparation (BigQuery).
2. Model Execution (Google Sheets: LINEST).
3. Prediction Result.

I. DATA PREPARATION & CLEANING

Centralizes data cleaning and ID standardization

- TABLE RENTALS

```
bqproj-435911.zen_city.rentals:
trip_id, subscriber_type, bike_id, bike_type, start_time, start_station_id,
start_station_name, end_station_id, end_station_name, duration_minutes

-- 1. CHECKING POTENTIAL PRIMARY KEY:
select count(*) as total_rows,
count (distinct(trip_id)) as trip_id,
count(distinct (bike_id)) as bike_id,
count(distinct subscriber_type) as subscriber_type,
count(distinct bike_type) as bike_type,
count(distinct start_time) as start_time,
count(distinct start_station_id) as start_station_id,
count(distinct start_station_name) as start_station_name,
count(distinct end_station_id) as end_station_id,
count(distinct end_station_name) as end_station_name,
count(distinct duration_minutes) as duration_minutes
from `bqproj-435911.zen_city.rentals`;

-- 2. CHECKING MISSING VALUES
select sum(case when subscriber_type is null then 1 else 0 end) as subscriber_type,
sum(case when bike_id is null then 1 else 0 end) as bike_id,
sum(case when bike_type is null then 1 else 0 end) as bike_type,
sum(case when start_time is null then 1 else 0 end) as start_time,
sum(case when start_station_id is null then 1 else 0 end) as start_station_id,
sum(case when start_station_name is null then 1 else 0 end) as start_station_name,
sum(case when end_station_id is null then 1 else 0 end) as end_station_id,
sum(case when end_station_name is null then 1 else 0 end) as end_station_name,
sum(case when duration_minutes is null then 1 else 0 end) as duration_minutes
from `bqproj-435911.zen_city.rentals`;
--> ONE STATION IN end_station_id HAS NO ID

-- 3. ENSURE THAT trip_id HAS NO NULL VALUES
select count(trip_id) as null_trip_id
from `bqproj-435911.zen_city.rentals`
where trip_id is null;
--> NO null VALUES


-- 4. VERIFY IF THERE ARE NULL VALUES IN start_station_id
select count(start_station_id) as start_station_id
```

```sql
from `bqproj-435911.zen_city.rentals`
where start_station_id is null;
--> NO null VALUES

-- 5. CHECKING NULL VALUES IN end_station_id
select *
from `bqproj-435911.zen_city.rentals`
where end_station_id is null;
--> STATION WITHOUT ID: end_station_name: Springfest 2022

-- 6. CHECKING IF THE STATION WITHOUT ID: Springfest 2022 ITS PRESENT AT TABLE station_info
select *
from bqproj-435911.zen_city.station_info
where name like '%pringfest%';
--> THERE'S NO DATA TO DISPLAY (STATION Springfest 2022 NOT PRESENT AT TABLE station_info)

-- 7. Due to a single null value in the end_station_id column, we decided to analyze it
independently of end_station_name to ensure result accuracy.
select distinct(end_station_id) as end_station_id
from `bqproj-435911.zen_city.rentals`
--> 81 STATIONS

select distinct(end_station_name) as end_station_name
from `bqproj-435911.zen_city.rentals`
order by 1;
--> 82 STATIONS

select distinct end_station_id, end_station_name
from `bqproj-435911.zen_city.rentals`
order by 1;
--> 83 STATIONS...I realized that something is wrong and decided to check:

-- 8. IDENTIFYING ROW COUNT DISCREPANCIES BETWEEN end_station_id AND end_station_name.
select end_station_id,
      count(distinct end_station_name) as count_end_station_name
from `bqproj-435911.zen_city.rentals`
group by end_station_id
having count(distinct end_station_name) > 1
      or end_station_id is null;
--> RESULTS: TWO STATION ID WHERE EACH ONE HAS TWO NAMES
-- 4057 - 6th/Chalmers (The stations have the same name, but one contains a trailing space
character.)
-- 2563 Rainey/Davis AND Rainey/Driskill
-- null 1
```

```
-- 9. CHECKING Rainey STATION IN station_info TABLE.
select *
from `bqproj-435911.zen_city.station_info`
where name like '%aine%';
--> 'Rainey/Driskill' is found in the station_info table with the address 698 Davis St, suggesting
that these might be the same station.

-- 10. IDENTIFY STATIONS IN THE RENTALS TABLE THAT ARE NOT RECORDED IN THE STATION_INFO TABLE.
select distinct (safe_cast(end_station_id as INT64)) as end_station_id, r.end_station_name,
      i.station_id
from `bqproj-435911.zen_city.rentals` r
left join `bqproj-435911.zen_city.station_info` i
    on safe_cast(end_station_id as INT64) = i.station_id
where i.station_id is null;
```

RESUME FINDINGS:

```
--> POTENTIAL PRIMARY KEY: trip_id.
--> start_station_id AND end_station_id CAN BE USED TO JOIN BOTH TABLES(rentals + station_info)
--> end_station_id:
   1. SAVED AS STRING
   2. HAS ONE NULL VALUE: Station 'Springfest 2022' HAS NO ID, IS NOT PRESENT AT TABLE
stations_info AND HAVE ONLY ONE TRIP
   3. TWO STATIONS ID WHERE EACH ONE HAS TWO RECORDED NAMES:
      4057 - 6th/Chalmers (The stations have the same name, but one contains a trailing space
character.)
      2563 - Rainey/Davis AND Rainey/Driskill ('Rainey/Driskill' is found in the station_info table
with the address 698 Davis St, suggesting that these might be the same station.)
--> SINCE WE ARE JOINING THE TABLES ON end_station_id, the end_station_name VALUES DO NOT REQUIRE
CLEANING OR MODIFICATION.
--> TOTAL end_stations: 80 STATIONS (WITH THE NO ID STATION INCLUDED)
```

- TABLE STATION_INFO

```
bqproj-435911.zen_city.station_info
station_id, name, status, location, address, alternate_name, city_asset_number, property_type,
number_of_docks, power_type, footprint_length, footprint_width, notes    council_district,
image, modified_date

-- 1. CHECKING POTENTIAL PRIMARY KEY
select count(distinct station_id) as rows_station_id,
  count(distinct name) as name,
  count(distinct status) as status,
  count(distinct location) as location,
```

```sql
  count(distinct address) as address,
  count(distinct alternate_name) as alternate_name,
  count(distinct city_asset_number) as city_asset_number,
  count(distinct property_type) as property_type,
  count(distinct number_of_docks) as number_of_docks,
  count(distinct power_type) as power_type,
  count(distinct footprint_length) as footprint_length,
  count(distinct footprint_width) as footprint_width,
  count(distinct notes) as notes,
  count(distinct council_district) as council_district,
  count(distinct image) as image,
  count(distinct modified_date) as modified_date
from `bqproj-435911.zen_city.station_info`;
--> TOTAL 101 STATIONS

-- 2. CHECKING MISSING VALUES
select sum(case when station_id is null then 1 else 0 end) as station_id,
sum(case when name is null then 1 else 0 end) as name,
sum(case when status is null then 1 else 0 end) as status,
sum(case when location is null then 1 else 0 end) as location,
sum(case when address is null then 1 else 0 end) as address,
sum(case when alternate_name is null then 1 else 0 end) as alternate_name,
sum(case when city_asset_number is null then 1 else 0 end) as city_asset_number,
sum(case when property_type is null then 1 else 0 end) as property_type,
sum(case when number_of_docks is null then 1 else 0 end) as number_of_docks,
sum(case when power_type is null then 1 else 0 end) as power_type,
sum(case when footprint_length is null then 1 else 0 end) as footprint_length,
sum(case when footprint_width is null then 1 else 0 end) as footprint_width,
sum(case when notes is null then 1 else 0 end) as notes,
sum(case when council_district is null then 1 else 0 end) as council_district,
sum(case when image is null then 1 else 0 end) as image,
sum(case when modified_date is null then 1 else 0 end) as modified_date,
from `bqproj-435911.zen_city.station_info`;
--> TEN COLUMNS WITH NULL VALUES:
-- address, alternate_name, city_asset_number, property_type, number_of_docks, power_type,
footprint_length,  footprint_width, notes, image.

-- 3. IDENTIFY STATIONS IN THE RENTALS TABLE THAT ARE NOT RECORDED IN THE STATION_INFO TABLE.
select distinct (safe_cast(end_station_id as INT64)) as end_station_id, r.end_station_name,
      i.station_id
from `bqproj-435911.zen_city.rentals` r
left join `bqproj-435911.zen_city.station_info` i
    on safe_cast(end_station_id as INT64) = i.station_id
where i.station_id is null;
```

end_station_id	end_station_name	station_id
7188	22nd/Pearl	null
7131	13th/Trinity	null
4938	22.5/Rio Grande	null
7125	23rd/San Gabriel	null
7190	Rio Grande/12th	null
null	Springfest 2022	null
7187	South Congress/Mary	null

```
-- 4. ANALYSING STATIONS status:
select status,
  count(status) as total_status
from `bqproj-435911.zen_city.station_info`
group by status;
--> CLOSED 24
--> ACTIVE 77

# ACTIVE STATIONS:
select *
from `bqproj-435911.zen_city.station_info`
where status = 'active';

# CLOSED STATIONS:
select *
from `bqproj-435911.zen_city.station_info`
where status = 'closed';

select *
from `bqproj-435911.zen_city.station_info`
where city_asset_number = 16729
--> station: 2502  Barton Springs & Riverside(closed) moved to 3293  East 2nd & Pedernales(active)

-- 5. STATUS OF THE STATIONS FROM RENTALS TABLE
select i.status,
    count(distinct end_station_id) as end_station_id
from `bqproj-435911.zen_city.rentals` r
left join `bqproj-435911.zen_city.station_info` i
  on safe_cast(end_station_id as int64) = i.station_id
group by i.status;
----> active  73
----> closed  1
----> null  6
```

```
# IDENTIFYING NULL VALUES
select distinct end_station_id , i.status
from `bqproj-435911.zen_city.rentals` r
left join `bqproj-435911.zen_city.station_info` i
  on safe_cast(end_station_id as int64) = i.station_id
order by i.status;
--> The null values correspond to stations present in the RENTALS table but missing from the
station_info table.

# IDENTIFYING THE 'closed' STATION
select distinct end_station_id , i.status
from `bqproj-435911.zen_city.rentals` r
left join `bqproj-435911.zen_city.station_info` i
  on safe_cast(end_station_id as int64) = i.station_id
where i.status like 'closed'
order by i.status;
--> station 3455

# IDENTIFYING IF STATION 3455 HAS TRIPS
select end_station_id, count(trip_id) as total_trips
from `bqproj-435911.zen_city.rentals`
where end_station_id = '3455'
group by end_station_id
--> station 3455 has 48 total_trips

-- 6. ANALYSING property_type
# TYPES OF property_type
select DISTINCT property_type
from `bqproj-435911.zen_city.station_info`;
--> 5 TYPES OF property_type: parkland, sidewalk, paid_parking, nonmetered_parking,
undetermined_parking

# MOST POPULAR property_type
select property_type,
  count(property_type) as total_property_type
from `bqproj-435911.zen_city.station_info`
where property_type is not null
group by property_type
order by total_property_type DESC;
```

property_type	total_property_type
sidewalk	28
undetermined_parking	20
paid_parking	19
nonmetered_parking	8
parkland	6

```
-- 7. CHECKING RELATIONS BETWEEN name AND alternate_name
select station_id,
    name,
    alternate_name
from `bqproj-435911.zen_city.station_info`
where alternate_name is not null;
--> STATIONS WITH ALTERNATE NAME:  2
-- 2574 | Zilker Park  | Zilker Park at Barton Springs and William Burton Drive
-- 3619 | 6th & Congress  | Congress & 6th Street

-- 8. TOTAL STATIONS PER council_district
select council_district,
  count(council_district) as total_stations_district,
  round((count(station_id) * 100) / sum(count(station_id)) over(), 1) as percentage
from `bqproj-435911.zen_city.station_info`
where council_district is not null
group by council_district
order by percentage DESC;
```

council_district	total_stations_district	percentage
9	59	58.4
1	16	15.8
3	16	15.8
5	5	5.0
8	3	3.0
10	2	2.0

```
-- 9. ANALYSING power_type
select power_type,
   count(power_type) as total_stations
from `bqproj-435911.zen_city.station_info`
group by power_type;
--> solar - 78
--> non-metered - 3
```

```
# power_type OF THE MOST POPULAR STATION 2498.
select *
from `bqproj-435911.zen_city.station_info`
where station_id = 2498
--> non-metered

-- 10. ANALYSING number_of_docks
# MODE number_of_docks = 13
select number_of_docks,
 count(number_of_docks) as mode_of_docks,
from `bqproj-435911.zen_city.station_info`
where number_of_docks is not null

group by number_of_docks
order by mode_of_docks DESC;

# AVG number_of_docks = 13.58
select avg(number_of_docks) as avg_number_of_docks
from `bqproj-435911.zen_city.station_info`
where number_of_docks is not null;
```

FINDINGS:

--> POTENTIAL PRIMARY KEY: station_id

--> TOTAL 101 STATIONS

--> TEN COLUMNS WITH NULL VALUES

--> We identified a potential match between station 2545 ACC - Rio Grande & 12th and the end_station_name 7190 Rio Grande/12th (similar name and only 220 meters apart). However, since the latter was used only three times, we did not change the ID.

--> STATION: 2502 Barton Springs & Riverside(closed) moved to 3293 East 2nd & Pedernales(active)

--> STATION: 3455 appears as 'closed' but has 48 total_trips in the rentals table.

--> The majority of stations utilize a solar power type; however, the most popular station is non-metered.

station_info		rentals		FINDINGS
station_id	name	end_station_id	end_station_name	
3841	23rd & Rio Grande	4938	22.5/Rio Grande	IS ESSENTIAL FOR ANALYSIS BUT IS NOT PRESENT IN THE station_info TABLE. WE NEED TO USE THE AVG_DOCK VALUE. BOTH STATIONS ARE NEAR TO EACH OTHER(220 meters away). STATION 3841 APPEARS TO BE CLOSED.
111	23rd & San Gabriel	7125	23rd/San Gabriel	SAME NAME AND HAS THIS NOTE:(Kiosk ID and Footprint length & width to be revised.)
1111	13th & Trinity	7131	13th/Trinity	SAME NAME AND HAS THIS NOTE:(The Kiosk ID and footprint length & width still need to be revised.)

0	South Congress/Mary	7187	South Congress/Mary	SAME NAME AND HAS THIS NOTE: (In the gutter)
3792	22nd & Pearl	7188	22nd/Pearl	BOTH STATION HAVE THE SAME NAME AND ARE AT THE SAME LOCATION AND APPEARS IN BOTH TABLES WITH DIFFERENT IDs.
2545	ACC-Rio Grande & 12th	7190	Rio Grande/12th	STATION 2545 APPEARS TO BE CLOSED AND HAS NO NUMBERS OF DOCKS. WE NEED TO USE THE AVG_DOCK VALUE. BOTH STATIONS ARE NEAR TO EACH OTHER(220 meters away)

```
--> STATIONS 7125, 7131, 7187 AND 7188 WILL BE MAPPED TO THEIR CORRECT station_id:
case when end_station_id = '7187' then 0
       when end_station_id = '7188' then 3792
       when end_station_id = '7131' then 1111
       when end_station_id = '7125' then 111
       else safe_cast(end_station_id as INT64) end as clean_end_station_id
```

II. DEMAND PATTERNS AND CUSTOMER PROFILING

This section provides the foundational diagnostic analysis, detailing the primary customer segments, quantifying spatial demand concentration, and evaluating asset utilization and core trip duration characteristics.

```
-- 1. START STATIONS IN rentals TABLE (start_station_id):
select distinct start_station_id, start_station_name
from `bqproj-435911.zen_city.rentals`;
--> 7 STATIONS

-- 2. TOP START STATIONS BY USAGE PERCENTAGE
select distinct start_station_id,
          start_station_name,
          count(trip_id) as total_rentals,
          round((count(trip_id) * 100.) / sum(count(trip_id)) over(), 1) as percentage
from `bqproj-435911.zen_city.rentals`
group by start_station_id,
         start_station_name
order by total_rentals desc;
```

Row	start_station_id	start_station_name	total_rentals	percentage
1	2498	Dean Keeton/Speedway	3001	27.8
2	2547	21st/Guadalupe	2299	21.3
3	3797	21st/University	1579	14.6
4	3799	23rd/San Jacinto @ DKR Stadium	1380	12.8
5	4938	22.5/Rio Grande	968	9.0
6	2566	Electric Drive/Sandra Muraida Way @ Pfluger Ped Brid...	822	7.6
7	3660	East 6th/Medina	731	6.8

```
-- 3. TOP END STATIONS BY USAGE PERCENTAGE
select distinct safe_cast(end_station_id as INT64) as end_station_id, end_station_name,
              count(trip_id) as total_rentals,
          round((count(trip_id) * 100) / sum(count(trip_id)) over(), 1) as percentage
from `bqproj-435911.zen_city.rentals`
group by end_station_id, end_station_name
order by total_rentals desc;
```

(The visualization displays only the top 15 end stations.)

Row	end_station_id	end_station_name	total_rentals	percentage
1	3798	21st/Speedway @ PCL	1864	17.3
2	3838	26th/Nueces	1153	10.7
3	3795	Dean Keeton/Whitis	805	7.5
4	7125	23rd/San Gabriel	700	6.5
5	7188	22nd/Pearl	616	5.7
6	2498	Dean Keeton/Speedway	602	5.6
7	3793	28th/Rio Grande	592	5.5
8	2547	21st/Guadalupe	524	4.9
9	2548	Guadalupe/West Mall @ University Co-op	446	4.1
10	3799	23rd/San Jacinto @ DKR Stadium	346	3.2
11	4938	22.5/Rio Grande	290	2.7
12	3797	21st/University	247	2.3
13	2566	Electric Drive/Sandra Muraida Way @ Pfluger Ped Bridge	159	1.5
14	3792	22nd/Pearl	133	1.2
15	3660	East 6th/Medina	103	1.0

```
-- 4. Only seven distinct start_stations were found, and every one of them is also recorded as an
end station. To verify this, we decided to be sure and to check...
```

```sql
with start_distinct as (
  select distinct start_station_id, start_station_name
  from `bqproj-435911.zen_city.rentals`
  where start_station_id is not null
),
end_distinct as (
  select distinct end_station_id
  from `bqproj-435911.zen_city.rentals`
  where end_station_id is not null
)
select
 s.start_station_id, s.start_station_name
from start_distinct s
inner join end_distinct e
  on s.start_station_id = safe_cast(end_station_id as INT64);
--> ALL 7 START STATIONS ARE ALSO END STATION.

-- 5. ANALYSING subscriber_type:
select distinct subscriber_type
from `bqproj-435911.zen_city.rentals`;
--> TOTAL OF SUBSCRIBERS 11 TYPES



-- 6. TYPES AND SUM OF SUBSCRIBERS
select subscriber_type,
    count(subscriber_type) as users_subscriber_type,
    round((count(trip_id) * 100) / sum(count(trip_id)) over(), 1) AS percentage_of_total_trips
from `bqproj-435911.zen_city.rentals`
where subscriber_type is not null
group by subscriber_type
order by users_subscriber_type desc;
```

Row	subscriber_type	users_subscriber_type	percentage_of_total_trips
1	Student Membership	7838	72.7
2	Local365	835	7.7
3	Local31	814	7.6
4	Pay-as-you-ride	356	3.3
5	Explorer	344	3.2
6	3-Day Weekender	205	1.9
7	24 Hour Walk Up Pass	138	1.3
8	Single Trip (Pay-as-you-ride)	133	1.2
9	U.T. Student Membership	113	1.0
10	HT Ram Membership	2	0.0
11	Annual Membership	2	0.0

```
-- 7. Bike_type ANALYSIS BY USAGE PERCENTAGE
with total_bikes as (
 select bike_type,
   count(bike_type) as count_bike_type,
   sum(count(bike_type)) over() as total_trips
 from `bqproj-435911.zen_city.rentals`
 where bike_type is not null
 group by bike_type
)
select
 bike_type,
 count_bike_type,
round((count_bike_type * 100) / total_trips, 1) AS percentage
from total_bikes;
```

Row	bike_type	count_bike_type	percentage
1	electric	9341	86.7
2	classic	1439	13.3

```
-- 8. BIKE ID ANALYSIS: COUNT OF UNIQUE BIKES AND TOTAL TRIPS.
select bike_id,
  count(bike_id) as total_trips
from `bqproj-435911.zen_city.rentals`
group by bike_id
order by total_trips DESC;
--> 570 bikes have been utilized.
```

```
select bike_id,
count(bike_id) as total_trips
from `bqproj-435911.zen_city.rentals`
group by bike_id
having count(bike_id) <= 1;
--> 55 bikes were used only one time.

-- 9. ANALYSIS OF LOW-USAGE BIKES (1-2 Trips)
with bike_utilization as (
      select bike_id,
            count(bike_id) as total_trips
from `bqproj-435911.zen_city.rentals`
group by bike_id
),
metrics as (
    select count(b.bike_id) as total_bikes_utilized,
        sum(case when b.total_trips = 1 then 1 else 0 end) as bikes_used_only_once,
        sum(case when b.total_trips <= 2 then 1 else 0 end) as bikes_used_twice_or_less
from bike_utilization as b
)
select m.total_bikes_utilized,
    m.bikes_used_only_once,
    bikes_used_twice_or_less,
    round((m.bikes_used_only_once * 100) / nullif(m.total_bikes_utilized, 0), 1) as
percentage_used_only_once,
    round((m.bikes_used_twice_or_less * 100) / nullif(m.total_bikes_utilized, 0), 1) as
percentage_used_twice_or_less
from metrics as m;
```

total_bikes_utilized	bikes_used_twice_or_less	percentage_used_only_once	percentage_used_twice_or_less
570	97	9.6	17.0

```
-- 10. CALCULATING THE AVERAGE BIKE USAGE
with bikes as (select bike_id,
  count(bike_id) as total_trips
from `bqproj-435911.zen_city.rentals`
group by bike_id
order by total_trips DESC)

select avg(sum_trips) as total
from bikes;
--> RESULT: 18.91

-- 11. EXPLORATION OF TRIPS DURATION (duration_minutes):
```

```
# MODE duration_minutes
select duration_minutes,
    count(duration_minutes) as total_trips,
    round((count(duration_minutes) * 100) / sum(count(duration_minutes)) over(), 1) as percentage
from `bqproj-435911.zen_city.rentals`
group by duration_minutes
order by total_trips DESC;
```

Row	duration_minutes	total_trips	percentage
1	4	1518	14.1
2	5	1414	13.1
3	3	1344	12.5
4	6	1001	9.3
5	7	731	6.8
6	2	704	6.5
7	8	474	4.4
8	9	332	3.1
9	10	232	2.2
10	11	180	1.7

```
--> This result indicates that most trips are less than 15 minutes.

# CENTRAL TENDENCY AND DISPERSION METRICS
select count(duration_minutes) as rows_duration_minutes,
    sum(duration_minutes) as sum_duration_minutes,
    avg(duration_minutes) as avg_duration_minutes,
    max(duration_minutes) as max_duration_minutes,
    min(duration_minutes) as min_duration_minutes
from `bqproj-435911.zen_city.rentals`;
```

rows_duration_mi...	sum_duration_mi...	avg_duration_min...	max_duration_min...	min_duration_min...
10780	237818	22.06103896103...	4874	2

```
-- AVG = 22,06min
-- OUTLIERS: MAX = 4874min   MIN = 2min

# ANALYSING THE OUTLIERS
--> Only one trip with the MAX duration = 4874 min
select count(trip_id) as total_trips
from `bqproj-435911.zen_city.rentals`
where duration_minutes = 4874;

--> 704 trips with the MIN duration = 2 min
select count(trip_id) as total_trips
from `bqproj-435911.zen_city.rentals`
```

```sql
where duration_minutes = 2;

-- 12. ANALYSING MOST POPULAR start_stations AND THEIR CHARACTERISTICS:
with rentals as (
select
  trip_id, start_station_id, start_station_name, subscriber_type, bike_id, bike_type,
start_time
from `bqproj-435911.zen_city.rentals`
where start_station_id is not null
),
bike_usage as (
  select
    start_station_id,
    bike_type,
    count(trip_id) as trips_by_bike_type
  from rentals
  group by start_station_id,
    bike_type
),
docks as (
  select
   r.start_station_id,
   r.start_station_name,
   i.number_of_docks,
   round(count(r.trip_id) / count(distinct date(r.start_time)),1) as avg_trips_per_day,
   i.property_type,
   i.footprint_length,
   i.footprint_width,
   count(r.trip_id) as total_trips,
   i.council_district
 from rentals r
 inner join `bqproj-435911.zen_city.station_info` i
   on r.start_station_id = i.station_id
 group by r.start_station_id,
   r.start_station_name,
   i.number_of_docks,
   i.property_type,
   i.footprint_length,
   i.footprint_width,
   i.council_district
 order by total_trips DESC
)
select
   d.start_station_id,
```

```
   d.start_station_name,
   d.number_of_docks,
   d.total_trips,
   d.avg_trips_per_day,
   b.bike_type,
   b.trips_by_bike_type,
   round((b.trips_by_bike_type * 100) / d.total_trips, 1) AS percentage_of_trips,
   d.property_type,
   (d.footprint_length * d.footprint_width) as size_dock,
   d.council_district,
from docks d
left join bike_usage b
  on d.start_station_id = b.start_station_id
order by d.total_trips DESC,
   d.start_station_id,
   b.trips_by_bike_type DESC;
--> #1 2498 4th/Sabine or Dean Keeton/Speedway    --> 3001 trips  - 17 docks (different
name at table station_info: 4th/Sabine)
    #2 2547 21st/Guadalupe                       --> 2299 trips - 13 docks
    #3 3797 21st/University                      --> 1579 trips - 19 docks
    #4 3799 23rd/San Jacinto @ DKR Stadium       --> 1380 trips - 22 docks
    #5 4938 22.5/Rio Grande                      --> 968 trips  - no information about docks,
using the avg_docks
    #6 2566 Electric Drive/Sandra Muraida Way @ Pfluger Ped Bridge  --> 822 - 19 docks
    #7 3660 East 6th/Medina                      --> 731 - 11 docks

-- 13. ANALYSING MOST POPULAR end_stations AND THEIR CHARACTERISTICS:
with rentals as (
select
  trip_id, subscriber_type, bike_id, bike_type, start_time,
    case when end_station_id = '7187' then 0
       when end_station_id = '7188' then 3792
       when end_station_id = '7131' then 1111
       when end_station_id = '7125' then 111
       else safe_cast(end_station_id as INT64)
    end as clean_end_station_id , end_station_name
from `bqproj-435911.zen_city.rentals`
where end_station_id is not null
),
bike_usage as (
  --  new CTE to calculate the use of each type bike per station
  select
    clean_end_station_id,
    bike_type,
```

```
    count(trip_id) as trips_by_bike_type,
  from rentals
  group by clean_end_station_id,
    bike_type
),
docks as (
  select
   r.end_station_name,
   r.clean_end_station_id,
   i.number_of_docks,
   round(count(r.trip_id) / count(distinct date(r.start_time)),1) as avg_trips_per_day,
   i.property_type,
   i.footprint_length,
   i.footprint_width,
   count(r.trip_id) as total_trips,
   i.council_district
 from rentals r
 inner join `bqproj-435911.zen_city.station_info` i
   on r.clean_end_station_id = i.station_id
 group by r.clean_end_station_id,
   r.end_station_name,
   i.number_of_docks,
   i.property_type,
   i.footprint_length,
   i.footprint_width,
   i.council_district
 order by total_trips DESC
)
select
   d.clean_end_station_id,
   d.end_station_name,
   d.number_of_docks,
   d.total_trips,
   d.avg_trips_per_day,
   b.bike_type,
   b.trips_by_bike_type,
   round((b.trips_by_bike_type * 100) / d.total_trips, 1) AS percentage_of_trips,
   d.property_type,
   (d.footprint_length * d.footprint_width) as size_dock,
   d.council_district
from docks d
left join bike_usage b
  on d.clean_end_station_id = b.clean_end_station_id
order by d.total_trips DESC,
```

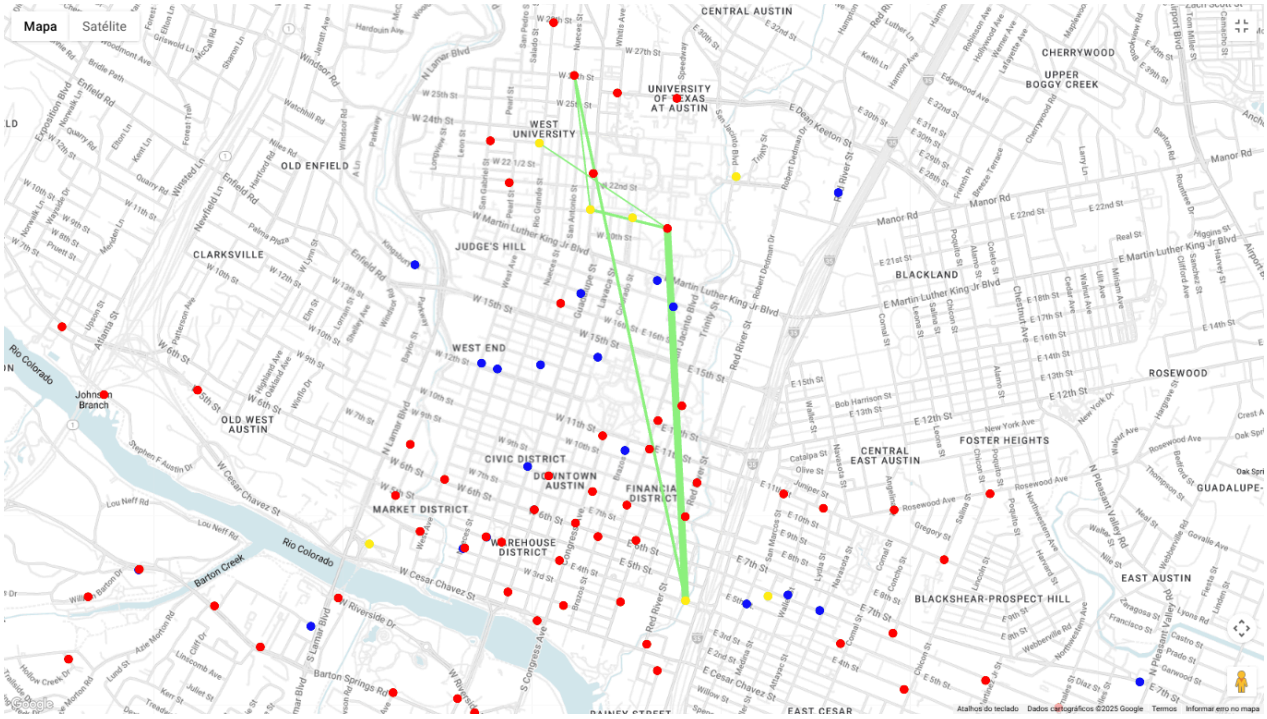
```
   d.clean_end_station_id,
   b.trips_by_bike_type DESC;
--> #1  3798  21st/Speedway @ PCL --> 1864 - 22 docks
    #2  3838  26th/Nueces         --> 1153 - 13 docks
    #3  3795  Dean Keeton/Whitis  --> 805  - 19 docks
    #4  3792  22nd/Pearl          --> 749  - 13 docks
    #5  111   23rd/San Gabriel    --> 700  - 13 docks
    #6  2498  Dean Keeton/Speedway--> 602  - 17 docks
    #7  3793  28th/Rio Grande     --> 592  - 13 docks

-- 14. POPULAR ROUTES:
with rentals as (
 select
  trip_id,start_station_id, start_station_name, start_time,
      case when end_station_id = '7187' then 0
       when end_station_id = '7188' then 3792
       when end_station_id = '7125' then 111
       when end_station_id = '7131' then 1111
       else safe_cast(end_station_id as INT64)
    end as end_station_id_clean ,
    end_station_name, duration_minutes
from `bqproj-435911.zen_city.rentals`
)
  select
    start_station_id,
    start_station_name,
    end_station_id_clean,
    end_station_name,
    count(trip_id) as total_trips,
    round((count(trip_id) * 100 / sum(count(trip_id)) over()), 1) as percentage
  from rentals
  group by start_station_id,
    start_station_name,
    end_station_id_clean,
    end_station_name
  order by total_trips DESC;
```

Row	start_station_id	start_station_name	end_station_i...	end_station_name	total_trips	percentage
1	2498	Dean Keeton/Speedway	3798	21st/Speedway @ PCL	831	7.7
2	2498	Dean Keeton/Speedway	3838	26th/Nueces	426	4.0
3	2547	21st/Guadalupe	3798	21st/Speedway @ PCL	415	3.8
4	2547	21st/Guadalupe	3838	26th/Nueces	316	2.9
5	4938	22.5/Rio Grande	3798	21st/Speedway @ PCL	296	2.7
6	2498	Dean Keeton/Speedway	3795	Dean Keeton/Whitis	267	2.5
7	2547	21st/Guadalupe	3792	22nd/Pearl	253	2.3
8	3799	23rd/San Jacinto @ DKR Stadium	3798	21st/Speedway @ PCL	233	2.2
9	3797	21st/University	3838	26th/Nueces	232	2.2
10	3797	21st/University	111	23rd/San Gabriel	229	2.1

```
-- 15. STATION LOCATION FROM MOST POPULAR ROUTES
with rentals as (
  select
    trip_id, subscriber_type, bike_id, bike_type, start_time,
    case when start_station_id = 4938 then 3841 else start_station_id end as
start_station_id_clean, -- Since both stations seems to be near to each other, for this case I
decided to use the location information of station 3841 to this query
       start_station_name,
    case when end_station_id = '7187' then 0
       when end_station_id = '7125' then 111
       when end_station_id = '7188' then 3792
       when end_station_id = '7131' then 1111
       else safe_cast(end_station_id as INT64)
    end as end_station_id_clean ,
    end_station_name, duration_minutes
  from `bqproj-435911.zen_city.rentals`
),
end_popular as (
  select
    end_station_id_clean,
    end_station_name,
    count(trip_id) as total_trips
  from rentals
  group by end_station_id_clean,
    end_station_name
)
select distinct end_station_id_clean,
     e.end_station_name,
     i.location,
     i.address,
     e.total_trips,
```

```
     i.number_of_docks
from end_popular e
left join `bqproj-435911.zen_city.station_info` i
    on e.end_station_id_clean = i.station_id
group by e.end_station_id_clean,
     e.end_station_name,
     i.location,
     i.address,
     i.number_of_docks,
     e.total_trips
order by e.total_trips DESC;

-- 16. Query to extract Latitude and Longitude for BigQuery Geo Viz mapping of the Top 5 Popular
Routes and location from all the stations:
with Location_Data as (
  -- CTE to Clean and Process Location Data, used by both queries
  select
    station_id,
    name, -- Adding the name for the point pop-up
    -- Converts the string (lat, lon) into numerical coordinates
    SAFE_CAST(SPLIT(TRIM(location, '()'), ', ')[OFFSET(1)] AS FLOAT64) AS longitude,
    SAFE_CAST(SPLIT(TRIM(location, '()'), ', ')[OFFSET(0)] AS FLOAT64) AS latitude
  FROM
    `bqproj-435911.zen_city.station_info`
  WHERE
    location IS NOT NULL AND TRIM(location) != ''
),
popular_routes AS (
  -- Logic to find the Top 5 Routes
  SELECT
    CASE
      WHEN start_station_id = 4938 THEN 3841 -- Since both stations seems to be near to each other,
for this case I decided to use the location information of station 3841 to this query
      ELSE start_station_id
    END AS start_station_id,
    safe_cast(end_station_id as INT64) as end_station_id,
    COUNT(trip_id) AS total_trips
  FROM
    `bqproj-435911.zen_city.rentals`
  GROUP BY
    1, 2
  ORDER BY
    total_trips DESC
  LIMIT 5
```

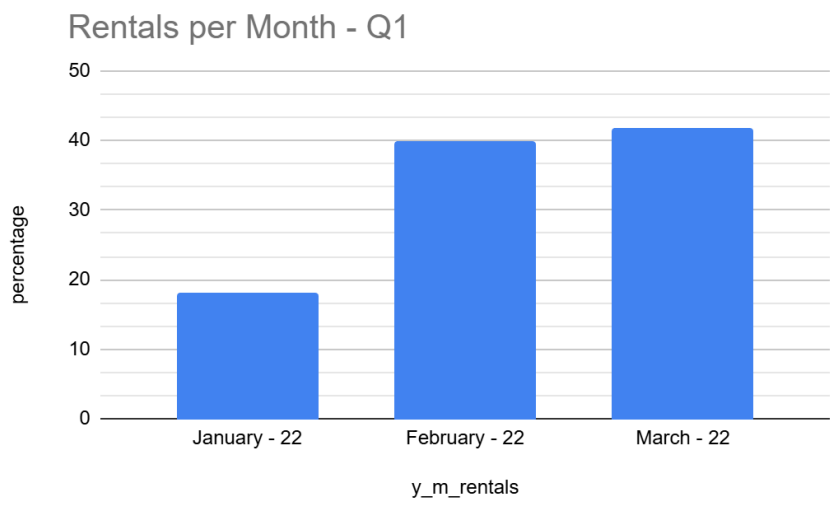
```sql
)
# SELECTION: ALL STATIONS (POINTS)
SELECT
  -- Creates the geographic point for the station
  ST_GEOGPOINT(t1.longitude, t1.latitude) AS geometria_visual,
  t1.name AS nome_elemento,
  'Estação' AS tipo_elemento,
  t1.station_id AS id_origem,
  NULL AS id_destino,
  NULL AS total_viagens
FROM
  Location_Data AS t1

UNION ALL

# SELECTION: THE TOP 5 ROUTES (LINES)
SELECT
  --- Creates the geographic line for the route
  ST_MAKELINE(
    ST_GEOGPOINT(s_loc.longitude, s_loc.latitude), -- Starting Point
    ST_GEOGPOINT(e_loc.longitude, e_loc.latitude)  -- Ending Point
  ) AS geometria_visual,
  -- Creates a descriptive name for the route pop-up
  CONCAT(s_loc.name, ' -> ', e_loc.name) AS nome_elemento,
  'Rota' AS tipo_elemento,
  t2.start_station_id AS id_origem,
  t2.end_station_id AS id_destino,
  t2.total_trips AS total_viagens
FROM
  popular_routes AS t2
INNER JOIN
  Location_Data AS s_loc -- Start Location
  ON t2.start_station_id = s_loc.station_id
INNER JOIN
  Location_Data AS e_loc -- End Location
  ON safe_cast(end_station_id as INT64) = e_loc.station_id;
```



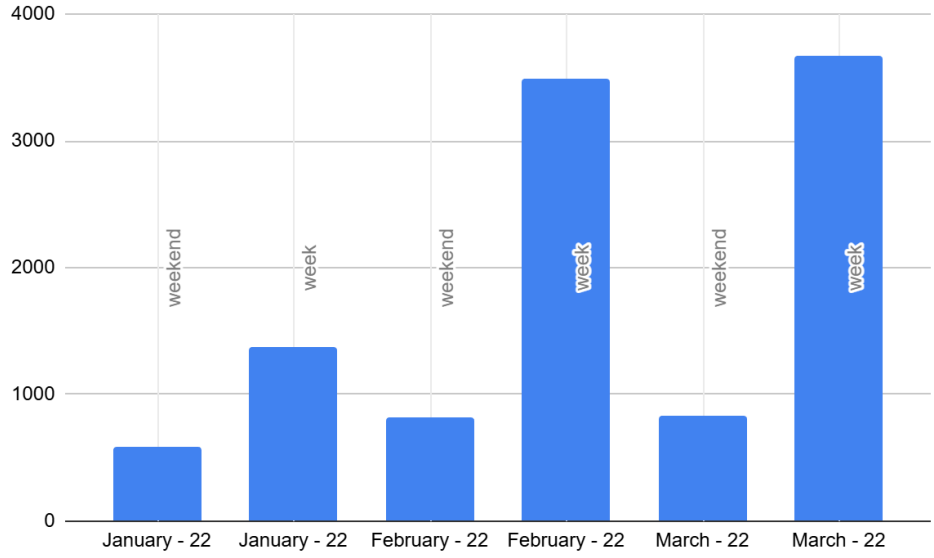
III. DEMAND AND TEMPORAL ANALYSIS

Queries focused on when and where demand is highest, guiding rebalancing strategies.

```
-- 1. TOTAL RENTALS PER MONTH
with rentals_per_month as
(select
format_date('%Y-%m', date(start_time)) as y_m_rentals,
count(trip_id) as total_rentals,
from `bqproj-435911.zen_city.rentals`
group by  y_m_rentals
order by  y_m_rentals)
select
y_m_rentals,
total_rentals,
round((total_rentals * 100) / sum(total_rentals) over (), 1) as percentage
from rentals_per_month;
```




```
-- 2. TOTAL RENTALS BY WEEK/WEEKEND PER MONTH
select
format_date('%Y-%m', date(start_time)) as y_m_rentals,
case when extract(dayofweek from start_time) = 1 or extract(dayofweek from start_time) = 7
then 'weekend' else 'week' end as day_of_week,
count(trip_id) as total_rentals
from `bqproj-435911.zen_city.rentals`
group by y_m_rentals,
 day_of_week
order by y_m_rentals,
 day_of_week DESC;
```




```
-- 3. TOTAL RENTALS BY DAY OF THE WEEK + WEEKDAY TYPE
select
 extract(dayofweek from start_time) as sequence_day,
 format_date('%A', date(start_time)) as week_day,
```

```
 case when extract(dayofweek from start_time) = 1 or extract(dayofweek from start_time) = 7
then 'weekend' else 'week' end as type_of_day,
 count(trip_id) as total_rentals
from `bqproj-435911.zen_city.rentals`
group by sequence_day,
     week_day,
     type_of_day
order by sequence_day;

-- 4. TOTAL OF RENTALS: WEEKDAY TYPE PER MONTH
select
 format_date('%Y-%m', date(start_time)) as y_m_rentals,
 extract(dayofweek from start_time) as sequence_day,
 format_date('%A', date(start_time)) as week_day,
 case when extract(dayofweek from start_time) = 1 or extract(dayofweek from start_time) = 7
then 'weekend' else 'week' end as type_of_day,
 count(trip_id) as total_rentals
from `bqproj-435911.zen_city.rentals`
group by y_m_rentals,
     sequence_day,
     week_day,
     type_of_day
order by y_m_rentals,
    sequence_day;

-- 5. TOTAL RENTALS PER HOUR (PEAK HOURS)
select
extract(hour from start_time) as hour_rental,
count(trip_id) as total_rentals
from `bqproj-435911.zen_city.rentals`
group by hour_rental
order by hour_rental ASC;

-- 6. TOTAL RENTALS WEEK/WEEKEND x HOUR
select
case when extract(dayofweek from start_time) = 1 or extract(dayofweek from start_time) = 7
then 'weekend' else 'week' end as day_of_week,
extract(hour from start_time) as hour_rental,
count(trip_id) as total_rentals
from `bqproj-435911.zen_city.rentals`
group by day_of_week,
  hour_rental
order by day_of_week,
  hour_rental desc;
```

```sql
# TOTAL RENTALS WEEK x HOUR
with rentals_day_hour as
(select
case when extract(dayofweek from start_time) = 1 or extract(dayofweek from start_time) = 7
then 'weekend' else 'week' end as day_of_week,
extract(hour from start_time) as hour_rental,
count(trip_id) as total_rentals
from `bqproj-435911.zen_city.rentals`
group by day_of_week,
  hour_rental
order by day_of_week,
  hour_rental desc)

select day_of_week,
      hour_rental,
      total_rentals
from rentals_day_hour
where day_of_week = 'week'
order by total_rentals DESC;

# TOTAL RENTALS WEEKEND x HOUR
with rentals_day_hour as
(select
case when extract(dayofweek from start_time) = 1 or extract(dayofweek from start_time) = 7
then 'weekend' else 'week' end as day_of_week,
extract(hour from start_time) as hour_rental,
count(trip_id) as total_rentals
from `bqproj-435911.zen_city.rentals`
group by day_of_week,
  hour_rental
order by day_of_week,
  hour_rental desc)

select day_of_week,
      hour_rental,
      total_rentals
from rentals_day_hour
where day_of_week = 'weekend'
order by total_rentals DESC;

-- 7. TOTAL RENTALS x POPULAR STATIONS x RUSH/PEAK HOURS (10hr - 19hr)
select
  extract(hour from start_time) as hour_rental,
```

```
  start_station_id,
  start_station_name,
  count(trip_id) as total_trips
from `bqproj-435911.zen_city.rentals`
where extract(hour from start_time) >= 10 and extract(hour from start_time) <= 19
group by hour_rental,
  start_station_id,
  start_station_name
order by start_station_id,hour_rental,
  total_trips DESC;

# ONLY THE 4 MOST POPULAR
select
  extract(hour from start_time) as hour_rental,
  start_station_id,
  start_station_name,
  count(trip_id) as total_trips
from `bqproj-435911.zen_city.rentals`
where extract(hour from start_time) >= 15 and extract(hour from start_time) <= 18
   and start_station_id in (2498, 2547, 3797, 3799)
group by hour_rental,
  start_station_id,
  start_station_name
order by start_station_id,
  hour_rental,
  total_trips DESC;


-- 8. start_stations: AVERAGE WEEK/WEEKEND DAY PER DOCK
   -- query to import start_stations data. decided to change the code of start_station 4938 that
wasn't in stations_info table to 3841 that is a station near to 4938 with closed status, to do not
loose the data of trip from station 4938.
with rentals as (
  select
   trip_id, subscriber_type, bike_id, bike_type, start_time,
   case when start_station_id = 4938 then 3841 else start_station_id  end as
clean_start_station_id,
   start_station_name, end_station_name,  duration_minutes
 from `bqproj-435911.zen_city.rentals`
 where start_station_id is not null
),
   -- query to extract type of day: week or weekend and total trips to each.
type_day_rentals as (
  select
```

```
     clean_start_station_id,
     case when extract(dayofweek from start_time) = 1 or extract(dayofweek from start_time) = 7
then 'weekend' else 'week' end as day_of_week,
      count(trip_id) as total_trips,
     count(distinct date(start_time)) as operation_days
  from rentals
  group by clean_start_station_id,
        day_of_week
),
station_data as (
    select
        station_id,
        name,
        number_of_docks
    from
        `bqproj-435911.zen_city.station_info`
)
select
    t1.clean_start_station_id,
    t2.name,
    t1.day_of_week,
    t2.number_of_docks,
    t1.total_trips,
    round((t1.total_trips / t1.operation_days) , 2) as avg_day_trip,
    round((t1.total_trips / t1.operation_days) / t2.number_of_docks, 2) as rental_day_per_dock
from type_day_rentals as t1
inner join station_data as t2
    on t1.clean_start_station_id = t2.station_id
where t2.number_of_docks is not null
    and t2.number_of_docks > 0
order by t1.clean_start_station_id,
    rental_day_per_dock desc;

-- 9. end_stations: AVERAGE WEEK/WEEKEND DAY PER DOCK
with rentals as (
  select
   trip_id, subscriber_type, bike_id, bike_type, start_time,
   case when end_station_id = '7125' then 111
       when end_station_id = '7187' then 0
       when end_station_id = '7188' then 3792
       when end_station_id = '7131' then 1111
       else safe_cast(end_station_id as INT64)
     end as clean_end_station_id,
   start_station_name, end_station_name,  duration_minutes
```

```
 from `bqproj-435911.zen_city.rentals`
 where end_station_id is not null
),
type_day_rentals as (
  select
     clean_end_station_id,
     case when extract(dayofweek from start_time) = 1 or extract(dayofweek from start_time) = 7
then 'weekend' else 'week' end as      day_of_week,
     count(trip_id) as total_trips,
     count(distinct date(start_time)) as operation_days
  from rentals
  group by clean_end_station_id,
        day_of_week
),
station_data as (
    select
        station_id,
        name,
        number_of_docks
    from
        `bqproj-435911.zen_city.station_info`
)
select
    t1.clean_end_station_id,
    t2.name,
    t1.day_of_week,
    t2.number_of_docks,
    t1.total_trips,
    round((t1.total_trips / t1.operation_days) , 2) as avg_day_trip,
    round((t1.total_trips / t1.operation_days) / t2.number_of_docks, 2) as rental_day_per_dock
from type_day_rentals as t1
inner join station_data as t2
    on t1.clean_end_station_id = t2.station_id
where t2.number_of_docks is not null
    and t2.number_of_docks > 0
order by avg_day_trip,
   t1.total_trips desc;

-- 10. TRIPS PER DOCK start_station
# Compares the trip volume to the station size, i.e., load relative to capacity.
with rentals as (
select
  trip_id, subscriber_type, bike_id, bike_type, start_time,
  case when start_station_id = 4938 then 3841 else start_station_id
```

```
    end as clean_start_station_id,
  start_station_name, end_station_name, duration_minutes
from `bqproj-435911.zen_city.rentals`
where start_station_id is not null
  or end_station_id is not null
)
select
    r.clean_start_station_id,
    r.start_station_name,
    round(count(r.trip_id) / i.number_of_docks,1) as trips_per_dock,
    round(count(r.trip_id) / count(distinct date(r.start_time)) / i.number_of_docks,1) AS
avg_daily_trips_per_dock
from rentals r
join `bqproj-435911.zen_city.station_info` i
   on r.clean_start_station_id = i.station_id
where number_of_docks is not null
  and number_of_docks > 0
group by r.clean_start_station_id,
  r.start_station_name,
  i.number_of_docks
order by avg_daily_trips_per_dock desc;

-- 11. TRIPS PER DOCK end_station
# Compares the trip volume to the station size, i.e., load relative to capacity.
with rentals as (
select
  trip_id, subscriber_type, bike_id, bike_type, start_time,
     case when end_station_id = '7125' then 111
       when end_station_id = '7187' then 0
       when end_station_id = '7188' then 3792
       when end_station_id = '7131' then 1111
       else safe_cast(end_station_id as INT64)
     end as clean_end_station_id,
  start_station_name, end_station_name, duration_minutes
from `bqproj-435911.zen_city.rentals`
where start_station_id is not null
  or end_station_id is not null
)
select
    r.clean_end_station_id,
    r.end_station_name,
    round(count(r.trip_id) / i.number_of_docks,1) as trips_per_dock,
    round(count(r.trip_id) / count(distinct date(r.start_time)) / i.number_of_docks,1) AS
avg_daily_trips_per_dock
```

```
from rentals r
join `bqproj-435911.zen_city.station_info` i
   on r.clean_end_station_id = i.station_id
where number_of_docks is not null
  and number_of_docks > 0
group by r.clean_end_station_id,
  r.end_station_name,
  i.number_of_docks
order by avg_daily_trips_per_dock desc;

-- 12. BIKE ROTATION ANALYSIS PER HOUR IN POPULAR ROUTES
-- OBJECTIVE: To calculate the exact hourly bike inventory at top stations for predictive
rebalancing.
RATIONALE FOR EXCLUSION:
This detailed Net Flow analysis (Departures vs. Arrivals) was developed to determine the hourly
inventory imbalance.
However, the model requires the initial starting inventory (bikes at the station) at the beginning
of the day (e.g., 00:00 AM) to accurately calculate the cumulative inventory level (Bike Count =
Start Inventory + Net Flow).
Since the project data does not provide this initial inventory count, the Net Flow metric alone
cannot reliably predict the exact moment a station will hit zero bikes (empty) or maximum capacity
(full).
CRITICAL INSIGHT: Despite this limitation, the Net Flow calculation was crucial as it provided
fundamental qualitative insight into the underlying stock dynamics of the network, confirming the
magnitude and direction of the hourly supply-demand imbalance.

# CALCULATING AND REMOVING OUTLIERS IN duration_minutes:
-- Finding de 99 percentil:
SELECT
    APPROX_QUANTILES(duration_minutes, 100)[OFFSET(99)] AS max_duration_minutes_p99
FROM
    `bqproj-435911.zen_city.rentals`
WHERE
    duration_minutes IS NOT NULL
    AND duration_minutes > 0;
    ---->RESULT: 393 minutes

# BIKE ROTATION ANALYSIS PER HOUR IN POPULAR ROUTES
# The first rows (with the highest negative net_flow_out value) indicate the stations that most
need free docks at that moment (need for bike removal).
WITH
  # DATA HANDLING AND TIME CALCULATION
  cleaned_rentals AS (
    SELECT
```

```
    trip_id,
    start_time,
    -- Apply Outlier Capping: 393 minutes
    CASE
      WHEN duration_minutes > 393 THEN 393
      ELSE duration_minutes
    END AS cleaned_duration_minutes,
    -- Calculate the end time
    TIMESTAMP_ADD(start_time, INTERVAL
      (CASE
          WHEN duration_minutes > 393 THEN 393
          ELSE duration_minutes
        END) MINUTE) AS end_time,
    -- Cleaning and Standardization of IDs
    CASE
      WHEN start_station_id = 4938 THEN 3841
      ELSE start_station_id
    END AS clean_start_station_id,
    start_station_name,
    CASE
      WHEN end_station_id = '4938' THEN 3841
      WHEN end_station_id = '7125' THEN 111
      WHEN end_station_id = '7187' THEN 0
      WHEN end_station_id = '7188' THEN 3792
      WHEN end_station_id = '7131' THEN 1111
      ELSE SAFE_CAST(end_station_id AS INT64)
    END AS clean_end_station_id,
    end_station_name

  FROM `bqproj-435911.zen_city.rentals`
  WHERE duration_minutes > 0
),
# IDENTIFICATION OF THE MOST POPULAR STATIONS (TOP N BY TOTAL FLOW)
top_stations AS (
    -- Calculates the total volume (departures + arrivals) for each station
    SELECT
        station_id,
        SUM(station_flow) as total_flow
    FROM (
        -- Departures
        SELECT clean_start_station_id as station_id, COUNT(trip_id) as station_flow
        FROM cleaned_rentals GROUP BY 1
        UNION ALL
        -- Arrivals
```

```
        SELECT clean_end_station_id as station_id, COUNT(trip_id) as station_flow
        FROM cleaned_rentals GROUP BY 1
    )
    GROUP BY station_id
    ORDER BY total_flow DESC
    LIMIT 25 -- We focus on the Top 25 busiest stations to prioritize rebalancing
),
# GENERATION OF HOURLY DEPARTURE AND ARRIVAL EVENTS PER STATION (FILTERING POPULAR ONES)
hourly_station_events AS (
  -- Departure Events (OUTFLOW: Rental)
  SELECT
    r.trip_id,
    r.clean_start_station_id AS station_id,
    r.start_station_name AS station_name,
    FORMAT_DATE('%Y-%m', DATE(r.start_time)) AS rental_month,
    FORMAT_DATE('%A', DATE(r.start_time)) AS day_of_week_name,
    EXTRACT(DAYOFWEEK FROM r.start_time) AS day_of_week_num, -- 1=Sun, 2=Mon...
    EXTRACT(HOUR FROM r.start_time) AS hour_of_day,
    1 AS departures,
    0 AS arrivals
  FROM cleaned_rentals r
  INNER JOIN top_stations ts ON r.clean_start_station_id = ts.station_id

  UNION ALL

  -- Arrival Events (INFLOW: Return)
  SELECT
    r.trip_id,
    r.clean_end_station_id AS station_id,
    r.end_station_name AS station_name,
    -- Uses the end time
    FORMAT_DATE('%Y-%m', DATE(r.end_time)) AS rental_month,
    FORMAT_DATE('%A', DATE(r.end_time)) AS day_of_week_name,
    EXTRACT(DAYOFWEEK FROM r.end_time) AS day_of_week_num,
    EXTRACT(HOUR FROM r.end_time) AS hour_of_day,
    0 AS departures,
    1 AS arrivals
  FROM cleaned_rentals r
  INNER JOIN top_stations ts ON r.clean_end_station_id = ts.station_id)
# FINAL GROUPING AND IMBALANCE CALCULATION
SELECT
    t1.rental_month,
    t1.day_of_week_name,
    t1.hour_of_day,
```

```sql
    t1.station_id,
    T1.station_name,
    -- Requested Columns
    SUM(t1.departures) AS total_rental_outflow,
    SUM(t1.arrivals) AS total_return_inflow,
    SUM(t1.departures - t1.arrivals) AS net_flow_out, -- KEY METRIC
    i.number_of_docks   -- Station Capacity for context
FROM hourly_station_events t1
LEFT JOIN `bqproj-435911.zen_city.station_info` i
    ON t1.station_id = i.station_id
GROUP BY 1, 2, 3, 4, 5, 9
ORDER BY
    t1.rental_month,
    t1.hour_of_day;

-- 13. Calculates the time-based station load, measuring average trip departures per dock, with
imputation for missing capacity data.
select
  r.start_station_id,
  s.name,
  EXTRACT(DAYOFWEEK FROM start_time) AS day_of_week,
  EXTRACT(HOUR FROM start_time) AS hour,
  COUNT(*) / COALESCE(number_of_docks, (SELECT AVG(number_of_docks) FROM
`bqproj-435911.zen_city.station_info`)) AS avg_load
FROM `bqproj-435911.zen_city.rentals` r
JOIN `bqproj-435911.zen_city.station_info` s
ON r.start_station_id = s.station_id
GROUP BY r.start_station_id, s.name, day_of_week, hour, s.number_of_docks;

-- 14. Query for the average hourly load during weekday peak hours, including the imputation
(normalization) of missing dock capacity for stations lacking station information.
WITH avg_docks AS (
  -- Calculates the average number of docks for stations with existing data.
  SELECT AVG(number_of_docks) AS value
  FROM `bqproj-435911.zen_city.station_info`
  WHERE number_of_docks IS NOT NULL
),
all_stations AS (
  -- All stations appearing in the rentals table.
  SELECT DISTINCT start_station_id, start_station_name
  FROM `bqproj-435911.zen_city.rentals`
),
station_filled AS (
  -- Joins station data and replaces missing number of docks with the overall average.
```

```sql
  SELECT
    s.start_station_id,
    s.start_station_name,
    COALESCE(si.number_of_docks, (SELECT value FROM avg_docks)) AS number_of_docks_filled
  FROM all_stations s
  LEFT JOIN `bqproj-435911.zen_city.station_info` si
    ON si.station_id = s.start_station_id
),
filtered_rentals AS (
  -- Filters only weekdays and hours 10–19 (inclusive).
  SELECT
    r.start_station_id,
    EXTRACT(DAYOFWEEK FROM r.start_time) AS day_of_week,
    EXTRACT(WEEK FROM r.start_time) AS week_number,
    COUNT(*) AS trips
  FROM `bqproj-435911.zen_city.rentals` r
  WHERE EXTRACT(DAYOFWEEK FROM r.start_time) BETWEEN 2 AND 6  -- Monday to Friday
    AND EXTRACT(HOUR FROM r.start_time) BETWEEN 10 AND 19      -- Hours 10–19 inclusive → 10 hours
  GROUP BY r.start_station_id, week_number, day_of_week
)
SELECT
  st.start_station_id,
  st.start_station_name,
  fr.week_number AS week,
  SUM(fr.trips) / 10.0 / st.number_of_docks_filled AS avg_trips_per_dock_hour_weekday
FROM filtered_rentals fr
LEFT JOIN station_filled st
  ON fr.start_station_id = st.start_station_id
GROUP BY st.start_station_id, st.start_station_name, week, st.number_of_docks_filled
ORDER BY st.start_station_id, week;
```

IV. PREDICTION MODEL (APRIL 1ST, STATION 2498)

This section details the methodology used to predict the total number of rentals for Station 2498 on April 1st, 2022 (Day 91 of the year), using a simple linear regression model to capture the overall growth trend observed during Q1 2022.

-- 1. Data Collection and Preparation (BigQuery)

The initial step involved preparing the historical demand data for the target station (ID 2498). We aggregated the number of trips daily, creating a time series dataset for the entire Q1 (90 days).

Query used for Filtering and Aggregation:

```sql
select count(trip_id) as total_rentals,
       extract(date from start_time) as date_rental,
from `bqproj-435911.zen_city.rentals`
```

```
where start_station_id = 2498
group by date_rental
order by total_rentals desc;
```

-- 2. Model Execution (Google Sheets: LINEST)

The results were exported from BigQuery to Google Sheets, where the LINEST function was applied to fit a simple linear model: Y = aX + b.

- Dependent Variable (Y): Daily Trip Count (total_rentals).
- Independent Variable (X): Day Number (ranging from 1 to 90).
- Target Prediction Day: April 1st, 2022, corresponds to Day $X = 91$.

The model coefficients derived were:

- Slope(a): 0.49 (Growth Rate)
- Y-Intercept(b): 12.35

-- 3. Prediction Result

The final prediction was calculated by substituting the target day (X=91) into the derived linear equation:

Y = 0.49 x 91 + 12.35
Predicted Rentals ~ 44.59 + 12.35
Predicted Rentals ~ 56.94 trips

Metric	Value	Interpretation
Prediction (Day 91)	~ 57 trips	The forecast based on the 90-day linear growth trend.
Prediction Error =STEYX()	25.07	High error suggests the model lacks features (e.g., day-of-week seasonality).